Архитектура ЭВМ и язык ассемблера

Семинар #35:

1. Аргументы и возвращаемые значения функций.
2. Рекурсивные функции.
3. Оптимизации потребления стека:
   хвостовая рекурсия, отказ от EBP.

20.03.2025

Аргументы и возвращаемые значения функций

32-битные числа в cdecl

```
__attribute__((cdecl))
uint32_t addOne32(uint32_t arg)
{
    return arg + 1U;
}
```

```
uint32_t u32 = addOne32(0U);
```

Передача аргумента:
размещение на стеке.

Возвращаемое значение:
регистр **EAX**.

```
addOne32:
push    ebp
mov     ebp, esp
mov     eax, dword [ebp + 0x8]
add     eax, 0x1
pop     ebp
ret
```

```
push    0x0
call    addOne32
add     esp, 0x4
mov     dword [ebp - 0x10], eax
```

64-битные числа в cdecl

```
__attribute__((cdecl))
uint64_t addOne64(uint64_t arg)
{
    return arg + 1U;
}
```

```
uint64_t u64  = addOne64(0U);
```

```
addOne64:
push    ebp
mov     ebp, esp
mov     eax, dword [ebp + 0x8]
mov     edx, dword [ebp + 0xC]
add     eax, 0x1
adc     edx, 0x0
pop     ebp
ret
```

Передача аргумента:
размещение на стеке.

Возвращаемое значение:
регистровая паре **EDX:EAX**.

```
push    0x0
push    0x0
call    addOne64
add     esp, 0x8
mov     dword [ebp - 0x18], eax
mov     dword [ebp - 0x14], edx
```

Структуры в cdecl

```c
typedef struct
{
    uint8_t array[8];
} Struct;

__attribute__((cdecl))
Struct addZeroStruct(Struct arg)
{
    return arg;
}
```

```
addZeroStruct:
push    ebp
mov     ebp, esp
mov     ecx, dword [ebp +  0x8]
mov     eax, dword [ebp +  0xC]
mov     edx, dword [ebp + 0x10]
mov     dword [ecx + 0x0], eax
mov     dword [ecx + 0x4], edx
mov     eax, dword [ebp + 0x8]
pop     ebp
retn    0x4
```

Передача аргумента: размещение на стеке.

Возвращаемое значение: запись по указателю, передаваемому первым аргументов.

Структуры в cdecl

```
Struct structure = {
    .array = {0, 1, 2, 3, 4, 5, 6, 7}
};

addZeroStruct(structure);
```

```
mov     dword [ebp - 0x20], 0x03020100
mov     dword [ebp - 0x1C], 0x07060504
lea     eax, [ebp - 0x30]
sub     esp, 0x4
push    dword [ebp - 0x1C]
push    dword [ebp - 0x20]
push    eax
call    addZeroStruct
add     esp, 0xC
```

Передача аргумента: размещение на стеке.

Возвращаемое значение: запись по указателю, передаваемому первым аргументов.

Рекурсивные функции

Алгоритм сортировки qsort

Алгоритм на псевдокоде:

```
quicksort(int A[],
  int low, int high)
{
  if (low >= high)
    return;

  int pivot =
    partition(A, low, high);

  quicksort(A, low, pivot - 1);
  quicksort(A, pivot + 1, high);
}
```

```
int partition(int A[], int low, int high)
{
  int p = A[high], i = low;
  for (int j = low; j < high; ++j)
  {
    if (A[j] <= p)
    {
      swap(&A[i], &A[j]);
      i += 1;
    }
  }
  swap(&A[i], &A[high]);
  return i;
}
```

Оптимизации потребления стека: хвостовая рекурсия, отказ от EBP

Хвостовая рекурсия

```c
unsigned isodd(unsigned n);

unsigned iseven(unsigned n)
{
    return (n == 0)? 1 : isodd(n - 1);
}

unsigned isodd(unsigned n)
{
    return (n == 0)? 0 : iseven(n - 1);
}

void test(void)
{
    printf("%d\n", iseven(10));
}
```

```asm
test:
push    ebp
mov     ebp, esp
sub     esp, 0x14
push    0xA
call    iseven
add     esp, 0xC
push    eax
push    format_str
push    0x1
call    printf
add     esp, 0x10
pop     ebp
ret
```

Первый аргумент `printf` –
кол-во форматных аргументов

Хвостовая рекурсия

```c
unsigned isodd(unsigned n);

unsigned iseven(unsigned n)
{
    return (n == 0)? 1 : isodd(n - 1);
}

unsigned isodd(unsigned n)
{
    return (n == 0)? 0 : iseven(n - 1);
}

void test(void)
{
    printf("%d\n", iseven(10));
}
```

```asm
iseven:
push    ebp
mov     ebp, esp
mov     eax, dword [ebp + 0x8]
test    eax, eax
jne     .L1
mov     eax, 0x1
pop     ebp
ret
.L1:
sub     eax, 0x1
mov     dword [ebp + 0x8], eax
pop     ebp
jmp     isodd
```

Флаг GCC **-foptimize-sibling-calls**

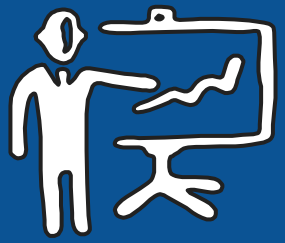
Отказ от EBP

```c
unsigned isodd(unsigned n);

unsigned iseven(unsigned n)
{
    return (n == 0)? 1 : isodd(n - 1);
}

unsigned isodd(unsigned n)
{
    return (n == 0)? 0 : iseven(n - 1);
}

void test(void)
{
    printf("%d\n", iseven(10));
}
```

```asm
iseven:
mov     eax, dword [esp + 0x4]
test    eax, eax
jne     .L1
mov     eax, 0x1
ret
.L1:
sub     eax, 0x1
mov     dword [esp + 0x4], eax
jmp     isodd
```

Флаг GCC **-fomit-frame-pointer**

Вопросы?



Красивые иконки взяты с сайта handdrawngoods.com